SATBAYEV UNIVERSITY

CSE6031 | Mathematics & Statistics | Computer Science

LABORATORY WORK 6

Continuous Probability Distributions Using Python

Uniform • Exponential • Normal • Computational Modeling

Purpose: connect the mathematical meaning of continuous probability distributions with computational modeling and visualization in Python.

Main tools: NumPy, SciPy, Matplotlib. Students should calculate probabilities analytically when possible and verify them computationally.

Learning outcomes

- Generate random samples from continuous distributions.
- Evaluate and interpret PDF and CDF values.
- Model Uniform, Exponential, and Normal distributions.
- Compare theoretical probabilities with simulation results.
- Connect distributions to realistic Computer Science problems.

Recommended workflow

Stage	What to do
1	Read the short theory and identify the distribution.
2	Write the mathematical model and parameters.
3	Implement the model in Python.
4	Visualize the distribution.
5	Compare simulation with the theoretical result.
6	Write a short interpretation.

SATBAYEV UNIVERSITY

CSE6031 | Mathematics & Statistics | Computer Science

1. Short Theory

Continuous random variable

A continuous random variable can take any real value in an interval. Probabilities are assigned to intervals, not to single exact points.

$$P(a \le X \le b) = \int_a^b f(x)\,dx$$

For a continuous variable, P(X = x) = 0. The PDF height is not itself a probability; probability is the area under the PDF curve.

PDF and CDF

Concept	Meaning	Formula
PDF	Describes probability density	f(x)
CDF	Probability that X is at most x	F(x)=P(X≤x)
Interval probability	Probability between two values	P(a<X≤b)=F(b)-F(a)

Python setup

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import uniform, expon, norm

np.random.seed(42)
```

Three distributions used in this lab

Distribution	Typical CS interpretation	Parameters
Uniform	Random value equally likely inside a fixed interval	a, b
Exponential	Waiting time until the next event	rate λ
Normal	Measurements centered around a mean	μ, σ

SATBAYEV UNIVERSITY

CSE6031 | Mathematics & Statistics | Computer Science

2. Task 1 — Uniform Distribution

A randomized algorithm chooses a delay uniformly between 2 and 8 milliseconds.

$$X \sim \text{Uniform}(2, 8)$$

$$f(x) = 1/(8-2) = 1/6, \text{ for } 2 \le x \le 8$$

Task: Find $P(3 \le X \le 5)$ analytically. Then simulate 10,000 delays and compare the simulated probability with the theoretical probability.

Step-by-step

1. The interval of interest has width $5-3=2$.
2. The total Uniform interval has width $8-2=6$.
3. Therefore $P(3 \le X \le 5) = 2/6 = 1/3 \approx 0.3333$.

Python

```
a, b = 2, 8
samples = np.random.uniform(a, b, 10000)

sim_prob = np.mean((samples >= 3) & (samples <= 5))
theory_prob = (5 - 3) / (b - a)

print("Theoretical:", theory_prob)
print("Simulation :", sim_prob)

x = np.linspace(a, b, 300)
pdf = uniform.pdf(x, loc=a, scale=b-a)

plt.plot(x, pdf)
plt.xlabel("Delay (ms)")
plt.ylabel("Density")
plt.title("Uniform Distribution: Random Delay")
plt.show()
```

Interpretation: the simulation should be close to 0.333. Small differences occur because simulation uses a finite number of random draws.

Student extension

Change the interval to 1–10 ms. Find and simulate $P(X > 7)$.

SATBAYEV UNIVERSITY

CSE6031 | Mathematics & Statistics | Computer Science

3. Task 2 — Exponential Distribution

A web service receives requests at an average rate of 4 requests per minute. Let X be the waiting time until the next request.

$$X \sim \text{Exponential}(\lambda = 4)$$

$$P(X \le t) = 1 - e^{(-\lambda t)}$$

Task: Find the probability that the next request arrives within 15 seconds.

Step-by-step solution

1. Convert 15 seconds to minutes: t = 15/60 = 0.25.

2. Use the exponential CDF.

$$P(X \le 0.25) = 1 - e^{(-4 \times 0.25)} = 1 - e^{(-1)} \approx 0.6321$$

So there is about a 63.2% chance that the next request arrives within 15 seconds.

Python

```
lam = 4
t = 15 / 60

theory = expon.cdf(t, scale=1/lam)
samples = np.random.exponential(scale=1/lam, size=10000)
sim = np.mean(samples <= t)

print("Theoretical:", theory)
print("Simulation :", sim)

x = np.linspace(0, 1.5, 400)
plt.plot(x, expon.pdf(x, scale=1/lam))
plt.xlabel("Waiting time (minutes)")
plt.ylabel("Density")
plt.title("Exponential Distribution: Request Waiting Time")
plt.show()
```

Computer Science meaning: Exponential models are useful for inter-arrival times, waiting times, service events, and reliability when the constant-rate assumption is reasonable.

Student extension

Find P(X > 30 seconds). Verify the result with simulation.

SATBAYEV UNIVERSITY

CSE6031 | Mathematics & Statistics | Computer Science

4. Task 3 — Normal Distribution

Suppose API response time is approximately Normally distributed with mean 120 ms and standard deviation 20 ms.

$$X \sim N(\mu=120, \sigma=20)$$

Task: Find the probability that a request takes less than 150 ms.

Step-by-step solution

1. Standardize the value using a z-score.

$$z = (x-\mu)/\sigma = (150-120)/20 = 1.5$$

2. Compute the standard Normal CDF at z=1.5.

3. $P(X<150) \approx 0.9332$.

Interpretation: about 93.3% of requests are expected to finish in under 150 ms if the Normal model is appropriate.

Python

```
mu = 120
sigma = 20
x0 = 150

prob = norm.cdf(x0, loc=mu, scale=sigma)
print("P(X < 150) =", prob)

x = np.linspace(50, 190, 400)
y = norm.pdf(x, loc=mu, scale=sigma)

plt.plot(x, y)
plt.axvline(x0, linestyle="--")
plt.xlabel("Response time (ms)")
plt.ylabel("Density")
plt.title("Normal Distribution: API Response Time")
plt.show()
```

Student extension

Find $P(100 \le X \le 140)$. Then calculate the z-scores for 100 and 140 and interpret them.

SATBAYEV UNIVERSITY

CSE6031 | Mathematics & Statistics | Computer Science

5. Task 4 — Compare Theory and Simulation

This task compares a theoretical probability with a Monte Carlo estimate.

> Model server latency as $X \sim N(80, 12^2)$. Estimate $P(X > 100)$ using both SciPy and 50,000 simulated observations.

```
mu = 80
sigma = 12
threshold = 100

theory = 1 - norm.cdf(threshold, loc=mu, scale=sigma)

samples = np.random.normal(mu, sigma, 50000)
simulation = np.mean(samples > threshold)

print("Theoretical:", theory)
print("Simulation :", simulation)
print("Absolute difference:", abs(theory - simulation))
```

Questions

1. Why will the simulated value not be exactly equal to the theoretical value?
2. What happens to the difference if the number of simulations increases from 1,000 to 100,000?
3. Would a Normal model always be appropriate for latency? Give one reason why not.

Expected observation

> As the number of simulated observations increases, the simulation generally stabilizes near the theoretical probability. This illustrates how computational simulation can validate an analytical probability model.

Optional challenge

> Repeat the experiment for sample sizes 100, 1,000, 10,000, and 100,000. Store the simulation error for each sample size and explain the pattern.

SATBAYEV UNIVERSITY

CSE6031 | Mathematics & Statistics | Computer Science

6. Final Applied Task

Choose ONE of the following Computer Science situations and model it using an appropriate continuous distribution.

Scenario	Suggested distribution
Random backoff time between 0 and 5 seconds	Uniform
Waiting time until the next network request	Exponential
Measurement noise around a sensor's expected value	Normal
Processing time of a stable service around a target	Normal

Your submission must contain: (1) definition of X, (2) selected distribution and parameters, (3) one probability question, (4) mathematical answer, (5) Python calculation, (6) one graph, and (7) 2–3 sentence interpretation.

Minimal Python template

```
# 1. Define parameters
# 2. Generate samples
# 3. Compute theoretical probability
# 4. Compute simulation probability
# 5. Plot the PDF or histogram
# 6. Print and interpret the result
```

Short assessment rubric

Criterion	Points
Correct probability model and parameters	2
Correct mathematical calculation	2
Working Python code	2
Clear visualization	1
Comparison of theory and simulation	1
Interpretation in CS context	2
Total	10

Key takeaways

- Uniform models equal density over a bounded interval.
- Exponential models waiting time under a constant event rate.
- Normal models continuous measurements centered around a mean.
- CDFs calculate interval/tail probabilities.
- Simulation is useful for checking and understanding theoretical probability results.